

Capgemini Most Asked Python DSA Questions

(Arrays, Strings, Stack, Queue, Trees, BST, Graphs, DP, Bit Manipulation)

Abhishek Rathor

Instagram id: SYNTAX ERROR (code.abhii07)

Key Features:

- **75+ Interview Questions** from Capgemini coding rounds.
- **Beginner to Advanced Topics:** Arrays → Strings → Stack → Queue → Trees → BST → Graphs → DP → Bit Manipulation.
- **Code in Python** with **Time & Space Complexity**.
- **Concept Explanations** included for quick revision.
- **Optimized Solutions** for interviews and campus placements.

Why This Guide?

- **Save Time:** No need to search multiple sources.
- **Practice Smart:** Focus on **most frequently asked questions**.
- **Learn Concepts:** Step-by-step explanations for each topic.
- **Placement Ready:** Covers all Capgemini rounds and coding challenges.

Success is not just about writing code, it's about solving problems.

ARRAYS (10 Questions)

Q1. Two Sum - Find pair with given sum

You are given an array of integers and a target sum. Find two numbers that add up to the target.

Solution:

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> mp;
    for(int i = 0; i < nums.size(); i++) {
        int complement = target - nums[i];
        if(mp.find(complement) != mp.end()) {
            return {mp[complement], i};
        }
        mp[nums[i]] = i;
    }
    return {};
}
```

Explanation: Uses hash map to store visited elements. For each element, checks if its complement exists in the map. Time complexity is $O(n)$ with $O(n)$ space.

Q2. Maximum Subarray Sum (Kadane's Algorithm)

You are given an array of integers. Find the contiguous subarray with the maximum sum.

Solution:

```
int maxSubArray(vector<int>& nums) {
    int maxSum = nums[0], currentSum = nums[0];
    for(int i = 1; i < nums.size(); i++) {
        currentSum = max(nums[i], currentSum + nums[i]);
        maxSum = max(maxSum, currentSum);
    }
    return maxSum;
}
```

Explanation: Maintains current sum and global maximum sum. At each position, decides whether to extend current subarray or start new one. Time complexity $O(n)$.

Q3. Rotate Array

You are given an array and an integer k. Rotate the array to the right by k steps.

Solution:

```
void rotate(vector<int>& nums, int k) {
    k = k % nums.size();
    reverse(nums.begin(), nums.end());
    reverse(nums.begin(), nums.begin() + k);
    reverse(nums.begin() + k, nums.end());
}
```

Explanation: Uses three reversals to achieve rotation. First reverses entire array, then first k elements, then remaining elements. Time $O(n)$, space $O(1)$.

Q4. Find Duplicate Number

You are given an array containing $n+1$ integers where each integer is between 1 and n . Find the duplicate number.

Solution:

```
int findDuplicate(vector<int>& nums) {
    int slow = nums[0], fast = nums[0];
    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while(slow != fast);
    slow = nums[0];
    while(slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }
    return slow;
}
```

Explanation: Uses Floyd's cycle detection algorithm treating array as linked list. Fast and slow pointers detect cycle, then finds cycle start which is the duplicate. $O(n)$ time, $O(1)$ space.

Q5. Merge Sorted Arrays

You are given two sorted arrays. Merge them into a single sorted array.

Solution:

```
void merge(vector<int>& nums1, int m, vector<int>& nums2, int n) {
    int i = m - 1, j = n - 1, k = m + n - 1;
    while(i >= 0 && j >= 0) {
        if(nums1[i] > nums2[j])
            nums1[k--] = nums1[i--];
        else
            nums1[k--] = nums2[j--];
    }
    while(j >= 0)
        nums1[k--] = nums2[j--];
}
```

Explanation: Merges from end to beginning to avoid overwriting elements. Compares elements from both arrays and places larger one at the end. Time $O(m+n)$, space $O(1)$.

Q6. Best Time to Buy and Sell Stock

You are given an array of stock prices. Find the maximum profit from one buy and one sell transaction.

Solution:

```
int maxProfit(vector<int>& prices) {
    int minPrice = INT_MAX, maxProfit = 0;
    for(int price : prices) {
        minPrice = min(minPrice, price);
        maxProfit = max(maxProfit, price - minPrice);
    }
    return maxProfit;
}
```

Explanation: Tracks minimum price seen so far and calculates profit at each point. Updates maximum profit if current profit is higher. Time $O(n)$, space $O(1)$.

Q7. Move Zeros to End

You are given an array with zeros and non-zeros. Move all zeros to the end maintaining relative order.

Solution:

```
void moveZeroes(vector<int>& nums) {
    int j = 0;
    for(int i = 0; i < nums.size(); i++) {
        if(nums[i] != 0) {
            swap(nums[i], nums[j]);
            j++;
        }
    }
}
```

Explanation: Uses two pointers where j tracks position for next non-zero element. Swaps non-zero elements to front maintaining order. Time $O(n)$, space $O(1)$.

Q8. Product of Array Except Self

You are given an array. Return array where each element is the product of all other elements.

Solution:

```
vector<int> productExceptSelf(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, 1);
    int left = 1, right = 1;
    for(int i = 0; i < n; i++) {
        result[i] *= left;
        left *= nums[i];
    }
    for(int i = n - 1; i >= 0; i--) {
        result[i] *= right;
        right *= nums[i];
    }
    return result;
}
```

Explanation: First pass calculates left products, second pass calculates right products and multiplies. Avoids division and handles zeros correctly. Time $O(n)$, space $O(1)$ excluding output.

Q9. Find Missing Number

You are given an array containing n distinct numbers from 0 to n . Find the missing number.

Solution:

```
int missingNumber(vector<int>& nums) {
    int n = nums.size();
    int expectedSum = n * (n + 1) / 2;
    int actualSum = 0;
    for(int num : nums)
        actualSum += num;
    return expectedSum - actualSum;
}
```

Explanation: Calculates expected sum using formula $n(n+1)/2$ and subtracts actual sum. The difference is the missing number. Time $O(n)$, space $O(1)$.

Q10. Maximum Product Subarray

You are given an array of integers. Find the contiguous subarray with the maximum product.

Solution:

```
int maxProduct(vector<int>& nums) {
    int maxProd = nums[0], minProd = nums[0], result = nums[0];
    for(int i = 1; i < nums.size(); i++) {
        if(nums[i] < 0) swap(maxProd, minProd);
        maxProd = max(nums[i], maxProd * nums[i]);
        minProd = min(nums[i], minProd * nums[i]);
        result = max(result, maxProd);
    }
    return result;
}
```

Explanation: Tracks both maximum and minimum products as negatives can become positives. Swaps when encountering negative number. Time $O(n)$, space $O(1)$.

STRINGS (10 Questions)

Q11. Valid Palindrome

You are given a string. Check if it is a palindrome considering only alphanumeric characters and ignoring case.

Solution:

```
bool isPalindrome(string s) {
    int left = 0, right = s.length() - 1;
    while(left < right) {
        while(left < right && !isalnum(s[left])) left++;
        while(left < right && !isalnum(s[right])) right--;
        if(tolower(s[left]) != tolower(s[right])) return false;
        left++; right--;
    }
    return true;
}
```

Explanation: Uses two pointers from both ends, skips non-alphanumeric characters. Compares characters after converting to lowercase. Time $O(n)$, space $O(1)$.

Q12. Longest Substring Without Repeating Characters

You are given a string. Find the length of the longest substring without repeating characters.

Solution:

```
int lengthOfLongestSubstring(string s) {
    unordered_map<char, int> mp;
    int maxLen = 0, start = 0;
    for(int end = 0; end < s.length(); end++) {
        if(mp.find(s[end]) != mp.end())
            start = max(start, mp[s[end]] + 1);
        mp[s[end]] = end;
        maxLen = max(maxLen, end - start + 1);
    }
    return maxLen;
}
```

Explanation: Uses sliding window with hash map to track last seen positions. Moves start pointer when duplicate found. Time $O(n)$, space $O(\min(m,n))$.

Q13. Valid Anagram

You are given two strings. Check if they are anagrams of each other.

Solution:

```
bool isAnagram(string s, string t) {
    if(s.length() != t.length()) return false;
    unordered_map<char, int> count;
    for(char c : s) count[c]++;
    for(char c : t) {
        if(--count[c] < 0) return false;
    }
    return true;
}
```

Explanation: Counts frequency of characters in first string and decrements for second string. If any count goes negative, not an anagram. Time $O(n)$, space $O(1)$.

Q14. Group Anagrams

You are given an array of strings. Group anagrams together.

Solution:

```
vector<vector<string>> groupAnagrams(vector<string>& strs) {
    unordered_map<string, vector<string>> mp;
    for(string s : strs) {
        string key = s;
        sort(key.begin(), key.end());
        mp[key].push_back(s);
    }
    vector<vector<string>> result;
    for(auto& p : mp)
        result.push_back(p.second);
    return result;
}
```

```
}
```

Explanation: Uses sorted string as key to group anagrams in hash map. All anagrams have same sorted form. Time $O(n \cdot k \log k)$ where k is max string length.

Q15. Longest Palindromic Substring

You are given a string. Find the longest palindromic substring.

Solution:

```
string longestPalindrome(string s) {
    int start = 0, maxLen = 0;
    auto expand = [&](int left, int right) {
        while(left >= 0 && right < s.length() && s[left] == s[right])
        {
            left--; right++;
        }
        if(right - left - 1 > maxLen) {
            maxLen = right - left - 1;
            start = left + 1;
        }
    };
    for(int i = 0; i < s.length(); i++) {
        expand(i, i); expand(i, i + 1);
    }
    return s.substr(start, maxLen);
}
```

Explanation: Expands around each center (odd and even length palindromes). Tracks longest palindrome found. Time $O(n^2)$, space $O(1)$.

Q16. String to Integer (atoi)

You are given a string. Convert it to a 32-bit signed integer following atoi rules.

Solution:

```
int myAtoi(string s) {
    int i = 0, sign = 1;
    long result = 0;
    while(i < s.length() && s[i] == ' ') i++;
    if(i < s.length() && (s[i] == '+' || s[i] == '-'))
        sign = (s[i++] == '-') ? -1 : 1;
    while(i < s.length() && isdigit(s[i])) {
        result = result * 10 + (s[i++] - '0');
        if(result * sign > INT_MAX) return INT_MAX;
        if(result * sign < INT_MIN) return INT_MIN;
    }
    return result * sign;
}
```

Explanation: Handles whitespace, sign, and digits while checking overflow. Returns clamped value if overflow occurs. Time $O(n)$, space $O(1)$.

Q17. Count and Say

You are given an integer n . Generate the n th term of the count-and-say sequence.

Solution:

```
string countAndSay(int n) {
    string result = "1";
    for(int i = 1; i < n; i++) {
        string next = "";
        int count = 1;
        for(int j = 1; j < result.length(); j++) {
            if(result[j] == result[j-1]) count++;
            else {
                next += to_string(count) + result[j-1];
                count = 1;
            }
        }
        next += to_string(count) + result.back();
        result = next;
    }
    return result;
}
```

Explanation: Iteratively builds sequence by counting consecutive digits. Appends count followed by digit to next term. Time $O(n*m)$ where m is length.

Q18. Reverse Words in String

You are given a string containing words separated by spaces. Reverse the order of words.

Solution:

```
string reverseWords(string s) {
    reverse(s.begin(), s.end());
    int n = s.length(), start = 0, end = 0;
    while(start < n) {
        while(start < n && s[start] == ' ') start++;
        if(start == n) break;
        if(end != 0) s[end++] = ' ';
        int wordStart = end;
        while(start < n && s[start] != ' ')
            s[end++] = s[start++];
        reverse(s.begin() + wordStart, s.begin() + end);
    }
    s.resize(end);
    return s;
}
```

Explanation: Reverses entire string, then reverses each word individually. Removes extra spaces in-place. Time $O(n)$, space $O(1)$.

Q19. Implement strStr (Find Needle in Haystack)

You are given two strings haystack and needle. Find the first occurrence of needle in haystack.

Solution:

```
int strStr(string haystack, string needle) {
    if(needle.empty()) return 0;
    int m = haystack.length(), n = needle.length();
    for(int i = 0; i <= m - n; i++) {
        int j = 0;
        while(j < n && haystack[i + j] == needle[j])
            j++;
    }
}
```

```

        if(j == n) return i;
    }
    return -1;
}

```

Explanation: Checks each position in haystack for match with needle. Returns index of first match or -1. Time $O(m*n)$, can optimize with KMP.

Q20. Longest Common Prefix

You are given an array of strings. Find the longest common prefix string.

Solution:

```

string longestCommonPrefix(vector<string>& strs) {
    if(strs.empty()) return "";
    string prefix = strs[0];
    for(int i = 1; i < strs.size(); i++) {
        int j = 0;
        while(j < prefix.length() && j < strs[i].length() &&
            prefix[j] == strs[i][j])
            j++;
        prefix = prefix.substr(0, j);
        if(prefix.empty()) break;
    }
    return prefix;
}

```

Explanation: Takes first string as prefix and compares with each string. Shrinks prefix when mismatch found. Time $O(S)$ where S is sum of all characters.

STACKS (8 Questions)

Q21. Valid Parentheses

You are given a string containing brackets. Check if brackets are properly closed and nested.

Solution:

```
bool isValid(string s) {
    stack<char> st;
    unordered_map<char, char> pairs = {{'}', '('}, {'}', '['}, {'}', ']'}, {'}', '{'}};
    for(char c : s) {
        if(pairs.find(c) != pairs.end()) {
            if(st.empty() || st.top() != pairs[c]) return false;
            st.pop();
        } else {
            st.push(c);
        }
    }
    return st.empty();
}
```

Explanation: Uses stack to match opening and closing brackets. Pushes opening brackets and pops when matching closing found. Time $O(n)$, space $O(n)$.

Q22. Min Stack

You are required to design a stack that supports push, pop, top, and retrieving minimum element in constant time.

Solution:

```
class MinStack {
    stack<int> st, minSt;
public:
    void push(int val) {
        st.push(val);
        if(minSt.empty() || val <= minSt.top())
            minSt.push(val);
    }
    void pop() {
        if(st.top() == minSt.top())
            minSt.pop();
        st.pop();
    }
    int top() { return st.top(); }
    int getMin() { return minSt.top(); }
};
```

Explanation: Uses two stacks, one for elements and one for tracking minimums. Min stack only stores values when new minimum is found. All operations $O(1)$.

Q23. Evaluate Reverse Polish Notation

You are given an array of strings representing arithmetic expression in Reverse Polish Notation. Evaluate it.

Solution:

```
int evalRPN(vector<string>& tokens) {
    stack<int> st;
    for(string& token : tokens) {
        if(token == "+" || token == "-" || token == "*" || token ==
"/") {
            int b = st.top(); st.pop();
            int a = st.top(); st.pop();
            if(token == "+") st.push(a + b);
            else if(token == "-") st.push(a - b);
            else if(token == "*") st.push(a * b);
            else st.push(a / b);
        } else {
            st.push(stoi(token));
        }
    }
    return st.top();
}
```

Explanation: Uses stack to store operands. When operator encountered, pops two operands and pushes result. Final stack top is answer. Time $O(n)$.

Q24. Daily Temperatures

You are given an array of temperatures. Return array where each element is number of days until warmer temperature.

Solution:

```
vector<int> dailyTemperatures(vector<int>& temperatures) {
```

```

    int n = temperatures.size();
    vector<int> result(n, 0);
    stack<int> st;
    for(int i = 0; i < n; i++) {
        while(!st.empty() && temperatures[i] >
temperatures[st.top()]) {
            int idx = st.top(); st.pop();
            result[idx] = i - idx;
        }
        st.push(i);
    }
    return result;
}

```

Explanation: Uses monotonic stack storing indices. When higher temperature found, pops and calculates days difference. Time $O(n)$, space $O(n)$.

Q25. Next Greater Element

You are given an array. For each element, find the next greater element to its right.

Solution:

```

vector<int> nextGreaterElements(vector<int>& nums) {
    int n = nums.size();
    vector<int> result(n, -1);
    stack<int> st;
    for(int i = 2 * n - 1; i >= 0; i--) {
        while(!st.empty() && st.top() <= nums[i % n])
            st.pop();
        if(i < n && !st.empty())
            result[i] = st.top();
        st.push(nums[i % n]);
    }
    return result;
}

```

Explanation: Traverses array twice (circular) from right using stack. Maintains decreasing monotonic stack. Time $O(n)$, space $O(n)$.

Q26. Largest Rectangle in Histogram

You are given heights of bars in histogram. Find the largest rectangle area.

Solution:

```

int largestRectangleArea(vector<int>& heights) {
    stack<int> st;
    int maxArea = 0, n = heights.size();
    for(int i = 0; i <= n; i++) {
        int h = (i == n) ? 0 : heights[i];
        while(!st.empty() && h < heights[st.top()]) {
            int height = heights[st.top()]; st.pop();
            int width = st.empty() ? i : i - st.top() - 1;
            maxArea = max(maxArea, height * width);
        }
        st.push(i);
    }
    return maxArea;
}

```

Explanation: Uses stack to track indices of increasing heights. Calculates area when smaller height encountered. Time $O(n)$, space $O(n)$.

Q27. Simplify Path

You are given an absolute Unix-style file path. Simplify it to canonical path.

Solution:

```

string simplifyPath(string path) {
    stack<string> st;
    stringstream ss(path);
    string token;
    while(getline(ss, token, '/')) {
        if(token == "" || token == ".") continue;
        if(token == "..") {
            if(!st.empty()) st.pop();
        } else {
            st.push(token);
        }
    }
    string result = "";
    while(!st.empty()) {
        result = "/" + st.top() + result;
        st.pop();
    }
    return result.empty() ? "/" : result;
}

```

```
}
```

Explanation: Splits path by '/' and uses stack to handle directory navigation. Handles '..' by popping and ignores '.'. Time O(n), space O(n).

Q28. Decode String

You are given an encoded string with pattern k[encoded_string]. Decode and return the string.

Solution:

```
string decodeString(string s) {
    stack<int> countSt;
    stack<string> strSt;
    string current = "";
    int k = 0;
    for(char c : s) {
        if(isdigit(c)) {
            k = k * 10 + (c - '0');
        } else if(c == '[') {
            countSt.push(k);
            strSt.push(current);
            current = "";
            k = 0;
        } else if(c == ']') {
            string temp = current;
            current = strSt.top(); strSt.pop();
            int count = countSt.top(); countSt.pop();
            while(count--) current += temp;
        } else {
            current += c;
        }
    }
    return current;
}
```

Explanation: Uses two stacks for counts and strings. Builds string and expands when closing bracket encountered. Time O(n), space O(n).

QUEUES (5 Questions)

Q29. Implement Queue using Stacks

You are required to implement a queue using only two stacks.

Solution:

```
class MyQueue {
    stack<int> s1, s2;
public:
    void push(int x) { s1.push(x); }
    int pop() {
        if(s2.empty()) {
            while(!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        int val = s2.top();
        s2.pop();
        return val;
    }
    int peek() {
        if(s2.empty()) {
            while(!s1.empty()) {
                s2.push(s1.top());
                s1.pop();
            }
        }
        return s2.top();
    }
    bool empty() { return s1.empty() && s2.empty(); }
};
```

Explanation: Uses input stack for enqueue and output stack for dequeue. Transfers elements when output stack empty. Amortized $O(1)$ for all operations.

Q30. Sliding Window Maximum

You are given an array and window size k. Find maximum in each sliding window.

Solution:

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> result;
    for(int i = 0; i < nums.size(); i++) {
        while(!dq.empty() && dq.front() <= i - k)
            dq.pop_front();
        while(!dq.empty() && nums[dq.back()] < nums[i])
            dq.pop_back();
        dq.push_back(i);
        if(i >= k - 1)
            result.push_back(nums[dq.front()]);
    }
    return result;
}
```

Explanation: Uses deque to maintain indices in decreasing order of values. Front always contains maximum of current window. Time $O(n)$, space $O(k)$.

Q31. Design Circular Queue

You are required to implement a circular queue with fixed size.

Solution:

```
class MyCircularQueue {
    vector<int> data;
    int front, rear, size, capacity;
public:
    MyCircularQueue(int k) : data(k), front(0), rear(-1), size(0),
    capacity(k) {}
    bool enqueue(int value) {
        if(isFull()) return false;
        rear = (rear + 1) % capacity;
        data[rear] = value;
        size++;
        return true;
    }
    bool dequeue() {
        if(isEmpty()) return false;
        front = (front + 1) % capacity;
        size--;
    }
}
```

```

        return true;
    }
    int Front() { return isEmpty() ? -1 : data[front]; }
    int Rear() { return isEmpty() ? -1 : data[rear]; }
    bool isEmpty() { return size == 0; }
    bool isFull() { return size == capacity; }
};

```

Explanation: Uses array with front and rear pointers using modulo for circular behavior. Tracks size separately for full/empty checks. All operations $O(1)$.

Q32. First Non-Repeating Character in Stream

You are given a stream of characters. Find first non-repeating character at each step.

Solution:

```

string firstNonRepeating(string s) {
    unordered_map<char, int> freq;
    queue<char> q;
    string result = "";
    for(char c : s) {
        freq[c]++;
        q.push(c);
        while(!q.empty() && freq[q.front()] > 1)
            q.pop();
        result += q.empty() ? '#' : q.front();
    }
    return result;
}

```

Explanation: Uses queue to maintain order and map for frequency. Removes repeated characters from queue front. Time $O(n)$, space $O(n)$.

Q33. Task Scheduler

You are given tasks and cooldown time n . Find minimum time to complete all tasks.

Solution:

```

int leastInterval(vector<char>& tasks, int n) {
    unordered_map<char, int> freq;
    int maxFreq = 0;
    for(char task : tasks) {
        freq[task]++;
        maxFreq = max(maxFreq, freq[task]);
    }
    int maxCount = 0;
    for(auto& p : freq)
        if(p.second == maxFreq) maxCount++;
    int time = (maxFreq - 1) * (n + 1) + maxCount;
    return max(time, (int)tasks.size());
}

```

Explanation: Calculates minimum intervals based on most frequent task. Formula accounts for cooldown periods between same tasks. Time $O(n)$, space $O(1)$.

RECURSION (8 Questions)

Q34. Fibonacci Number

You are given an integer n . Calculate the n th Fibonacci number using recursion.

Solution:

```

int fib(int n, unordered_map<int, int>& memo) {
    if(n <= 1) return n;
    if(memo.find(n) != memo.end()) return memo[n];
    return memo[n] = fib(n-1, memo) + fib(n-2, memo);
}

int fibonacci(int n) {
    unordered_map<int, int> memo;
    return fib(n, memo);
}

```

Explanation: Uses memoization to store computed values avoiding recomputation. Base cases are 0 and 1. Time $O(n)$ with memoization, space $O(n)$.

Q35. Power Function

You are given base x and exponent n . Calculate x^n using recursion efficiently.

Solution:

```
double myPow(double x, int n) {
    if(n == 0) return 1.0;
    if(n < 0) {
        x = 1 / x;
        n = -(n + 1);
        return x * myPow(x, n);
    }
    double half = myPow(x, n / 2);
    if(n % 2 == 0) return half * half;
    return x * half * half;
}
```

Explanation: Uses binary exponentiation dividing problem in half each step. Handles negative exponents by inverting base. Time $O(\log n)$, space $O(\log n)$.

Q36. Generate Parentheses

You are given n pairs of parentheses. Generate all combinations of well-formed parentheses.

Solution:

```
void generate(int open, int close, string current, vector<string>&
result) {
    if(open == 0 && close == 0) {
        result.push_back(current);
        return;
    }
    if(open > 0)
        generate(open - 1, close, current + "(", result);
    if(close > open)
        generate(open, close - 1, current + ")", result);
}
vector<string> generateParenthesis(int n) {
    vector<string> result;
    generate(n, n, "", result);
    return result;
}
```

Explanation: Recursively adds opening brackets when available and closing when valid. Tracks remaining open and close brackets. Time $O(4^n/\sqrt{n})$, space $O(n)$.

Q37. Letter Combinations of Phone Number

You are given a string containing digits 2-9. Return all possible letter combinations from phone keypad.

Solution:

```
void backtrack(string& digits, int idx, string current,
               vector<string>& result, vector<string>& mapping) {
    if(idx == digits.length()) {
        result.push_back(current);
        return;
    }
    string letters = mapping[digits[idx] - '0'];
    for(char c : letters)
        backtrack(digits, idx + 1, current + c, result, mapping);
}

vector<string> letterCombinations(string digits) {
    if(digits.empty()) return {};
    vector<string> mapping = {"", "", "abc", "def", "ghi", "jkl",
    "mno", "pqrs", "tuv", "wxyz"};
    vector<string> result;
    backtrack(digits, 0, "", result, mapping);
    return result;
}
```

Explanation: Uses backtracking to explore all combinations. Maps digits to letters and recursively builds strings. Time $O(4^n)$, space $O(n)$.

Q38. Permutations

You are given an array of distinct integers. Return all possible permutations.

Solution:

```
void backtrack(vector<int>& nums, int start, vector<vector<int>>&
result) {
    if(start == nums.size()) {
```

```

        result.push_back(nums);
        return;
    }
    for(int i = start; i < nums.size(); i++) {
        swap(nums[start], nums[i]);
        backtrack(nums, start + 1, result);
        swap(nums[start], nums[i]);
    }
}

vector<vector<int>> permute(vector<int>& nums) {
    vector<vector<int>> result;
    backtrack(nums, 0, result);
    return result;
}

```

Explanation: Swaps elements to generate permutations recursively. Backtracks by swapping back to original state. Time $O(n! \cdot n)$, space $O(n)$.

Q39. Subsets

You are given an array of unique integers. Return all possible subsets (power set).

Solution:

```

void backtrack(vector<int>& nums, int start, vector<int>& current,
               vector<vector<int>>& result) {
    result.push_back(current);
    for(int i = start; i < nums.size(); i++) {
        current.push_back(nums[i]);
        backtrack(nums, i + 1, current, result);
        current.pop_back();
    }
}

vector<vector<int>> subsets(vector<int>& nums) {
    vector<vector<int>> result;
    vector<int> current;
    backtrack(nums, 0, current, result);
    return result;
}

```

Explanation: Generates subsets by including or excluding each element. Explores all combinations recursively. Time $O(2^n)$, space $O(n)$.

Q40. Combination Sum

You are given array of candidates and target. Find all unique combinations that sum to target.

Solution:

```
void backtrack(vector<int>& candidates, int target, int start,
              vector<int>& current, vector<vector<int>>& result) {
    if(target == 0) {
        result.push_back(current);
        return;
    }
    if(target < 0) return;
    for(int i = start; i < candidates.size(); i++) {
        current.push_back(candidates[i]);
        backtrack(candidates, target - candidates[i], i, current,
result);
        current.pop_back();
    }
}
vector<vector<int>> combinationSum(vector<int>& candidates, int
target) {
    vector<vector<int>> result;
    vector<int> current;
    backtrack(candidates, target, 0, current, result);
    return result;
}
```

Explanation: Uses backtracking allowing same element multiple times. Prunes when target becomes negative. Time $O(n^{(t/m)})$, space $O(t/m)$.

Q41. Word Search

You are given a 2D board and a word. Check if word exists in the grid.

Solution:

```
bool dfs(vector<vector<char>>& board, string& word, int i, int j, int
idx) {
    if(idx == word.length()) return true;
    if(i < 0 || i >= board.size() || j < 0 || j >= board[0].size() ||
```

```

        board[i][j] != word[idx]) return false;
    char temp = board[i][j];
    board[i][j] = '#';
    bool found = dfs(board, word, i+1, j, idx+1) ||
                 dfs(board, word, i-1, j, idx+1) ||
                 dfs(board, word, i, j+1, idx+1) ||
                 dfs(board, word, i, j-1, idx+1);
    board[i][j] = temp;
    return found;
}

bool exist(vector<vector<char>>& board, string word) {
    for(int i = 0; i < board.size(); i++)
        for(int j = 0; j < board[0].size(); j++)
            if(dfs(board, word, i, j, 0)) return true;
    return false;
}

```

Explanation: Uses DFS with backtracking to explore all paths. Marks visited cells temporarily to avoid reuse. Time $O(mn4^L)$, space $O(L)$.

BINARY SEARCH TREES (7 Questions)

Q42. Validate Binary Search Tree

You are given root of binary tree. Check if it is a valid BST.

Solution:

```

bool isValidBST(TreeNode* root, long minVal = LONG_MIN, long maxVal =
LONG_MAX) {
    if(!root) return true;
    if(root->val <= minVal || root->val >= maxVal) return false;
    return isValidBST(root->left, minVal, root->val) &&
           isValidBST(root->right, root->val, maxVal);
}

```

Explanation: Recursively checks if each node value is within valid range. Left subtree must have values less than node, right greater. Time $O(n)$, space $O(h)$.

Q43. Lowest Common Ancestor of BST

You are given BST and two nodes. Find their lowest common ancestor.

Solution:

```
TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if(!root) return nullptr;
    if(root->val > p->val && root->val > q->val)
        return lowestCommonAncestor(root->left, p, q);
    if(root->val < p->val && root->val < q->val)
        return lowestCommonAncestor(root->right, p, q);
    return root;
}
```

Explanation: Uses BST property to navigate. LCA is node where paths to p and q diverge.
Time $O(h)$, space $O(h)$.

Q44. Kth Smallest Element in BST

You are given root of BST and integer k. Find the kth smallest element.

Solution:

```
void inorder(TreeNode* root, int& k, int& result) {
    if(!root) return;
    inorder(root->left, k, result);
    k--;
    if(k == 0) {
        result = root->val;
        return;
    }
    inorder(root->right, k, result);
}

int kthSmallest(TreeNode* root, int k) {
    int result = -1;
    inorder(root, k, result);
    return result;
}
```

Explanation: Performs inorder traversal which visits nodes in sorted order. Decrements k and returns when k becomes 0. Time $O(h+k)$, space $O(h)$.

Q45. Delete Node in BST

You are given root of BST and key. Delete node with given key.

Solution:

```
TreeNode* deleteNode(TreeNode* root, int key) {
    if(!root) return nullptr;
    if(key < root->val) {
        root->left = deleteNode(root->left, key);
    } else if(key > root->val) {
        root->right = deleteNode(root->right, key);
    } else {
        if(!root->left) return root->right;
        if(!root->right) return root->left;
        TreeNode* minNode = root->right;
        while(minNode->left) minNode = minNode->left;
        root->val = minNode->val;
        root->right = deleteNode(root->right, minNode->val);
    }
    return root;
}
```

Explanation: Finds node to delete, replaces with inorder successor if has two children. Handles cases of 0, 1, or 2 children. Time $O(h)$, space $O(h)$.

Q46. Convert Sorted Array to BST

You are given sorted array. Convert it to height-balanced BST.

Solution:

```
TreeNode* sortedArrayToBST(vector<int>& nums, int left, int right) {
    if(left > right) return nullptr;
    int mid = left + (right - left) / 2;
    TreeNode* root = new TreeNode(nums[mid]);
    root->left = sortedArrayToBST(nums, left, mid - 1);
    root->right = sortedArrayToBST(nums, mid + 1, right);
}
```

```

        return root;
    }
    TreeNode* sortedArrayToBST(vector<int>& nums) {
        return sortedArrayToBST(nums, 0, nums.size() - 1);
    }

```

Explanation: Recursively picks middle element as root ensuring balance. Left half becomes left subtree, right half becomes right subtree. Time $O(n)$, space $O(\log n)$.

Q47. Range Sum of BST

You are given BST root and range [low, high]. Find sum of values in range.

Solution:

```

int rangeSumBST(TreeNode* root, int low, int high) {
    if(!root) return 0;
    int sum = 0;
    if(root->val >= low && root->val <= high)
        sum += root->val;
    if(root->val > low)
        sum += rangeSumBST(root->left, low, high);
    if(root->val < high)
        sum += rangeSumBST(root->right, low, high);
    return sum;
}

```

Explanation: Uses BST property to prune search space. Only explores subtrees that can contain values in range. Time $O(n)$, space $O(h)$.

Q48. Insert into BST

You are given root of BST and value to insert. Insert and return modified tree.

Solution:

```

TreeNode* insertIntoBST(TreeNode* root, int val) {
    if(!root) return new TreeNode(val);
    if(val < root->val)
        root->left = insertIntoBST(root->left, val);
}

```

```

        else
            root->right = insertIntoBST(root->right, val);
        return root;
    }

```

Explanation: Recursively finds correct position using BST property. Creates new node when nullptr reached. Time $O(h)$, space $O(h)$.

TREES (9 Questions)

Q49. Maximum Depth of Binary Tree

You are given root of binary tree. Find its maximum depth.

Solution:

```

int maxDepth(TreeNode* root) {
    if(!root) return 0;
    return 1 + max(maxDepth(root->left), maxDepth(root->right));
}

```

Explanation: Recursively calculates depth of left and right subtrees. Returns maximum plus one for current node. Time $O(n)$, space $O(h)$.

Q50. Symmetric Tree

You are given root of binary tree. Check if it is symmetric around its center.

Solution:

```

bool isMirror(TreeNode* left, TreeNode* right) {
    if(!left && !right) return true;
    if(!left || !right) return false;
    return left->val == right->val &&
           isMirror(left->left, right->right) &&
           isMirror(left->right, right->left);
}

bool isSymmetric(TreeNode* root) {
    return !root || isMirror(root->left, root->right);
}

```

```
}
```

Explanation: Compares left and right subtrees recursively checking mirror property. Left's left must match right's right. Time $O(n)$, space $O(h)$.

Q51. Level Order Traversal

You are given root of binary tree. Return level order traversal of node values.

Solution:

```
vector<vector<int>> levelOrder(TreeNode* root) {
    if(!root) return {};
    vector<vector<int>> result;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()) {
        int size = q.size();
        vector<int> level;
        for(int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if(node->left) q.push(node->left);
            if(node->right) q.push(node->right);
        }
        result.push_back(level);
    }
    return result;
}
```

Explanation: Uses BFS with queue processing nodes level by level. Tracks level size to group nodes. Time $O(n)$, space $O(n)$.

Q52. Diameter of Binary Tree

You are given root of binary tree. Find length of longest path between any two nodes.

Solution:

```
int diameter = 0;
```

```

int height(TreeNode* root) {
    if(!root) return 0;
    int left = height(root->left);
    int right = height(root->right);
    diameter = max(diameter, left + right);
    return 1 + max(left, right);
}
int diameterOfBinaryTree(TreeNode* root) {
    diameter = 0;
    height(root);
    return diameter;
}

```

Explanation: Calculates height while tracking maximum path length through each node. Diameter is sum of left and right heights. Time $O(n)$, space $O(h)$.

Q53. Invert Binary Tree

You are given root of binary tree. Invert it (mirror reflection).

Solution:

```

TreeNode* invertTree(TreeNode* root) {
    if(!root) return nullptr;
    TreeNode* temp = root->left;
    root->left = invertTree(root->right);
    root->right = invertTree(temp);
    return root;
}

```

Explanation: Recursively swaps left and right children at each node. Base case returns nullptr for leaf children. Time $O(n)$, space $O(h)$.

Q54. Path Sum

You are given root and target sum. Check if there's root-to-leaf path summing to target.

Solution:

```

bool hasPathSum(TreeNode* root, int targetSum) {

```

```

    if(!root) return false;
    if(!root->left && !root->right)
        return root->val == targetSum;
    return hasPathSum(root->left, targetSum - root->val) ||
        hasPathSum(root->right, targetSum - root->val);
}

```

Explanation: Recursively subtracts node values from target. Returns true when leaf reached with remaining sum equal to leaf value. Time $O(n)$, space $O(h)$.

Q55. Lowest Common Ancestor of Binary Tree

You are given binary tree and two nodes. Find their lowest common ancestor.

Solution:

```

TreeNode* lowestCommonAncestor(TreeNode* root, TreeNode* p, TreeNode* q) {
    if(!root || root == p || root == q) return root;
    TreeNode* left = lowestCommonAncestor(root->left, p, q);
    TreeNode* right = lowestCommonAncestor(root->right, p, q);
    if(left && right) return root;
    return left ? left : right;
}

```

Explanation: Recursively searches for nodes in left and right subtrees. If found in both, current node is LCA. Time $O(n)$, space $O(h)$.

Q56. Binary Tree Right Side View

You are given root of binary tree. Return values visible from right side.

Solution:

```

vector<int> rightSideView(TreeNode* root) {
    vector<int> result;
    if(!root) return result;
    queue<TreeNode*> q;
    q.push(root);
    while(!q.empty()) {

```

```

        int size = q.size();
        for(int i = 0; i < size; i++) {
            TreeNode* node = q.front(); q.pop();
            if(i == size - 1) result.push_back(node->val);
            if(node->left) q.push(node->left);
            if(node->right) q.push(node->right);
        }
    }
    return result;
}

```

Explanation: Performs level order traversal and picks last node of each level. Uses BFS with queue. Time $O(n)$, space $O(n)$.

Q57. Serialize and Deserialize Binary Tree

You are required to design algorithm to serialize and deserialize binary tree.

Solution:

```

string serialize(TreeNode* root) {
    if(!root) return "null,";
    return to_string(root->val) + "," + serialize(root->left) +
serialize(root->right);
}

TreeNode* deserialize(string& data, int& idx) {
    if(idx >= data.length()) return nullptr;
    string val = "";
    while(data[idx] != ',') val += data[idx++];
    idx++;
    if(val == "null") return nullptr;
    TreeNode* root = new TreeNode(stoi(val));
    root->left = deserialize(data, idx);
    root->right = deserialize(data, idx);
    return root;
}

TreeNode* deserialize(string data) {
    int idx = 0;
    return deserialize(data, idx);
}

```

Explanation: Uses preorder traversal for serialization with null markers. Deserializes by parsing string recursively. Time $O(n)$, space $O(n)$.

GRAPHS (8 Questions)

Q58. Number of Islands

You are given 2D grid of '1's (land) and '0's (water). Count number of islands.

Solution:

```
void dfs(vector<vector<char>>& grid, int i, int j) {
    if(i < 0 || i >= grid.size() || j < 0 || j >= grid[0].size() ||
        grid[i][j] == '0') return;
    grid[i][j] = '0';
    dfs(grid, i+1, j);
    dfs(grid, i-1, j);
    dfs(grid, i, j+1);
    dfs(grid, i, j-1);
}

int numIslands(vector<vector<char>>& grid) {
    int count = 0;
    for(int i = 0; i < grid.size(); i++) {
        for(int j = 0; j < grid[0].size(); j++) {
            if(grid[i][j] == '1') {
                count++;
                dfs(grid, i, j);
            }
        }
    }
    return count;
}
```

Explanation: Uses DFS to mark all connected land cells. Each DFS call represents one island discovery. Time $O(mn)$, space $O(mn)$.

Q59. Clone Graph

You are given reference of node in connected undirected graph. Return deep copy.

Solution:

```
Node* cloneGraph(Node* node, unordered_map<Node*, Node*>& visited) {
    if(!node) return nullptr;
    if(visited.find(node) != visited.end()) return visited[node];
    Node* clone = new Node(node->val);
    visited[node] = clone;
    for(Node* neighbor : node->neighbors)
        clone->neighbors.push_back(cloneGraph(neighbor, visited));
    return clone;
}
Node* cloneGraph(Node* node) {
    unordered_map<Node*, Node*> visited;
    return cloneGraph(node, visited);
}
```

Explanation: Uses DFS with hash map to track cloned nodes. Creates copy and recursively clones neighbors. Time $O(n+e)$, space $O(n)$.

Q60. Course Schedule (Cycle Detection)

You are given number of courses and prerequisites. Check if you can finish all courses.

Solution:

```
bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {
    vector<vector<int>> graph(numCourses);
    vector<int> indegree(numCourses, 0);
    for(auto& pre : prerequisites) {
        graph[pre[1]].push_back(pre[0]);
        indegree[pre[0]]++;
    }
    queue<int> q;
    for(int i = 0; i < numCourses; i++)
        if(indegree[i] == 0) q.push(i);
    int count = 0;
    while(!q.empty()) {
        int course = q.front(); q.pop();
        count++;
        for(int next : graph[course]) {
            if(--indegree[next] == 0)
                q.push(next);
        }
    }
}
```

```

    }
    return count == numCourses;
}

```

Explanation: Uses Kahn's algorithm (topological sort) to detect cycles. If all courses processed, no cycle exists. Time $O(v+e)$, space $O(v+e)$.

Q61. Pacific Atlantic Water Flow

You are given $m \times n$ matrix representing heights. Find cells where water can flow to both oceans.

Solution:

```

void dfs(vector<vector<int>>& heights, vector<vector<bool>>& visited,
int i, int j, int prev) {
    if(i < 0 || i >= heights.size() || j < 0 || j >=
heights[0].size() ||
        visited[i][j] || heights[i][j] < prev) return;
    visited[i][j] = true;
    dfs(heights, visited, i+1, j, heights[i][j]);
    dfs(heights, visited, i-1, j, heights[i][j]);
    dfs(heights, visited, i, j+1, heights[i][j]);
    dfs(heights, visited, i, j-1, heights[i][j]);
}

vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights) {
    int m = heights.size(), n = heights[0].size();
    vector<vector<bool>> pacific(m, vector<bool>(n)), atlantic(m,
vector<bool>(n));
    for(int i = 0; i < m; i++) {
        dfs(heights, pacific, i, 0, -1);
        dfs(heights, atlantic, i, n-1, -1);
    }
    for(int j = 0; j < n; j++) {
        dfs(heights, pacific, 0, j, -1);
        dfs(heights, atlantic, m-1, j, -1);
    }
    vector<vector<int>> result;
    for(int i = 0; i < m; i++)
        for(int j = 0; j < n; j++)
            if(pacific[i][j] && atlantic[i][j])
                result.push_back({i, j});
}

```

```

        return result;
    }

```

Explanation: DFS from ocean edges inward checking reachability. Cells reachable from both oceans are added to result. Time $O(mn)$, space $O(mn)$.

Q62. Shortest Path in Binary Matrix

You are given $n \times n$ binary matrix. Find shortest clear path from top-left to bottom-right.

Solution:

```

int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
    if(grid[0][0] == 1) return -1;
    int n = grid.size();
    vector<vector<int>> dirs =
{{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
    queue<pair<int,int>> q;
    q.push({0, 0});
    grid[0][0] = 1;
    int steps = 1;
    while(!q.empty()) {
        int size = q.size();
        for(int i = 0; i < size; i++) {
            auto [x, y] = q.front(); q.pop();
            if(x == n-1 && y == n-1) return steps;
            for(auto& dir : dirs) {
                int nx = x + dir[0], ny = y + dir[1];
                if(nx >= 0 && nx < n && ny >= 0 && ny < n &&
grid[nx][ny] == 0) {
                    grid[nx][ny] = 1;
                    q.push({nx, ny});
                }
            }
            steps++;
        }
    }
    return -1;
}

```

Explanation: Uses BFS to find shortest path considering 8 directions. Marks visited cells to avoid revisiting. Time $O(n^2)$, space $O(n^2)$.

Q63. Rotting Oranges

You are given grid with fresh and rotten oranges. Find minimum time for all oranges to rot.

Solution:

```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int,int>> q;
    int fresh = 0;
    for(int i = 0; i < m; i++) {
        for(int j = 0; j < n; j++) {
            if(grid[i][j] == 2) q.push({i, j});
            if(grid[i][j] == 1) fresh++;
        }
    }
    if(fresh == 0) return 0;
    int minutes = 0;
    vector<vector<int>> dirs = {{-1,0},{1,0},{0,-1},{0,1}};
    while(!q.empty()) {
        int size = q.size();
        for(int i = 0; i < size; i++) {
            auto [x, y] = q.front(); q.pop();
            for(auto& dir : dirs) {
                int nx = x + dir[0], ny = y + dir[1];
                if(nx >= 0 && nx < m && ny >= 0 && ny < n &&
grid[nx][ny] == 1) {
                    grid[nx][ny] = 2;
                    fresh--;
                    q.push({nx, ny});
                }
            }
        }
        minutes++;
    }
    return fresh == 0 ? minutes - 1 : -1;
}
```

Explanation: Uses multi-source BFS starting from all rotten oranges. Spreads rot level by level tracking time. Time $O(mn)$, space $O(mn)$.

Q64. Word Ladder

You are given beginWord, endWord and wordList. Find shortest transformation sequence length.

Solution:

```
int ladderLength(string beginWord, string endWord, vector<string>& wordList) {
    unordered_set<string> dict(wordList.begin(), wordList.end());
    if(dict.find(endWord) == dict.end()) return 0;
    queue<string> q;
    q.push(beginWord);
    int level = 1;
    while(!q.empty()) {
        int size = q.size();
        for(int i = 0; i < size; i++) {
            string word = q.front(); q.pop();
            if(word == endWord) return level;
            for(int j = 0; j < word.length(); j++) {
                char original = word[j];
                for(char c = 'a'; c <= 'z'; c++) {
                    word[j] = c;
                    if(dict.find(word) != dict.end()) {
                        q.push(word);
                        dict.erase(word);
                    }
                }
                word[j] = original;
            }
        }
        level++;
    }
    return 0;
}
```

Explanation: Uses BFS trying all one-letter transformations at each level. Erases used words to avoid cycles. Time $O(m^2n)$, space $O(mn)$.

Q65. Graph Valid Tree

You are given n nodes and edges list. Check if edges form valid tree.

Solution:

```
bool validTree(int n, vector<vector<int>>& edges) {
    if(edges.size() != n - 1) return false;
    vector<int> parent(n);
    iota(parent.begin(), parent.end(), 0);
    function<int(int)> find = [&](int x) {
        return parent[x] == x ? x : parent[x] = find(parent[x]);
    };
    for(auto& edge : edges) {
        int p1 = find(edge[0]), p2 = find(edge[1]);
        if(p1 == p2) return false;
        parent[p1] = p2;
    }
    return true;
}
```

Explanation: Uses union-find to detect cycles. Valid tree has $n-1$ edges and no cycles. Time $O(n \cdot \alpha(n))$, space $O(n)$.

DYNAMIC PROGRAMMING (5 Questions)

Q66. Climbing Stairs

You are given n steps. Find number of ways to climb using 1 or 2 steps at a time.

Solution:

```
int climbStairs(int n) {
    if(n <= 2) return n;
    int prev2 = 1, prev1 = 2;
    for(int i = 3; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Explanation: Uses DP similar to Fibonacci. Ways to reach step i is sum of ways to reach $i-1$ and $i-2$. Time $O(n)$, space $O(1)$.

Q67. House Robber

You are given array of house values. Find maximum money you can rob without robbing adjacent houses.

Solution:

```
int rob(vector<int>& nums) {
    if(nums.empty()) return 0;
    int prev2 = 0, prev1 = 0;
    for(int num : nums) {
        int current = max(prev1, prev2 + num);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Explanation: At each house, chooses max between robbing current house plus i-2 or skipping it. Uses two variables for optimization. Time $O(n)$, space $O(1)$.

Q68. Coin Change

You are given coins array and amount. Find minimum coins needed to make amount.

Solution:

```
int coinChange(vector<int>& coins, int amount) {
    vector<int> dp(amount + 1, amount + 1);
    dp[0] = 0;
    for(int i = 1; i <= amount; i++) {
        for(int coin : coins) {
            if(i >= coin)
                dp[i] = min(dp[i], dp[i - coin] + 1);
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Explanation: Builds up solutions from 0 to amount. For each amount, tries all coins and picks minimum. Time $O(\text{amount} \times \text{coins})$, space $O(\text{amount})$.

Q69. Longest Increasing Subsequence

You are given array of integers. Find length of longest strictly increasing subsequence.

Solution:

```
int lengthOfLIS(vector<int>& nums) {
    vector<int> dp;
    for(int num : nums) {
        auto it = lower_bound(dp.begin(), dp.end(), num);
        if(it == dp.end())
            dp.push_back(num);
        else
            *it = num;
    }
    return dp.size();
}
```

Explanation: Uses binary search with patience sorting approach. Maintains array of smallest tail elements for each length. Time $O(n \log n)$, space $O(n)$.

Q70. Unique Paths

You are given $m \times n$ grid. Find number of unique paths from top-left to bottom-right.

Solution:

```
int uniquePaths(int m, int n) {
    vector<int> dp(n, 1);
    for(int i = 1; i < m; i++) {
        for(int j = 1; j < n; j++) {
            dp[j] += dp[j-1];
        }
    }
    return dp[n-1];
}
```

Explanation: Uses 1D DP array where each cell is sum of paths from left and top. Space optimized from 2D to 1D. Time $O(m*n)$, space $O(n)$.

BIT MANIPULATION (5 Questions)

Q71. Single Number

You are given array where every element appears twice except one. Find the single element.

Solution:

```
int singleNumber(vector<int>& nums) {
    int result = 0;
    for(int num : nums) {
        result ^= num;
    }
    return result;
}
```

Explanation: XOR of two same numbers is 0, and XOR of any number with 0 is number itself. All pairs cancel out. Time $O(n)$, space $O(1)$.

Q72. Number of 1 Bits (Hamming Weight)

You are given unsigned integer. Count number of '1' bits.

Solution:

```
int hammingWeight(uint32_t n) {
    int count = 0;
    while(n) {
        count++;
        n &= (n - 1);
    }
    return count;
}
```

Explanation: Brian Kernighan's algorithm flips rightmost set bit to 0 each iteration. Counts iterations until n becomes 0. Time $O(\log n)$, space $O(1)$.

Q73. Reverse Bits

You are given 32-bit unsigned integer. Reverse its bits.

Solution:

```
uint32_t reverseBits(uint32_t n) {
    uint32_t result = 0;
    for(int i = 0; i < 32; i++) {
        result <<= 1;
        result |= (n & 1);
        n >>= 1;
    }
    return result;
}
```

Explanation: Extracts least significant bit of n and adds to result. Shifts result left and n right 32 times. Time $O(1)$, space $O(1)$.

Q74. Power of Two

You are given integer n. Check if it is a power of two.

Solution:

```
bool isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Explanation: Power of 2 has only one bit set. n & (n-1) removes rightmost bit, result is 0 for powers of 2. Time $O(1)$, space $O(1)$.

Q75. Missing Number (Bit Manipulation)

You are given array containing n distinct numbers from 0 to n. Find missing number using XOR.

Solution:

```
int missingNumber(vector<int>& nums) {  
    int result = nums.size();  
    for(int i = 0; i < nums.size(); i++) {  
        result ^= i ^ nums[i];  
    }  
    return result;  
}
```

Explanation: XORs all indices and values. Duplicate numbers cancel out leaving only missing number.

Time $O(n)$, space $O(1)$.

 **Congratulations!**

You have successfully completed this DSA & Tech guide. This marks a step forward in your coding journey! Remember, learning never

stops—keep practicing, solving problems, and building projects. Every line of code you write brings you closer to becoming a master of Data Structures, Algorithms, and modern tech skills.

Stay connected with us for more **coding tutorials, challenges, tips, and tech content** on Instagram: **@SYNTAXERROR**. Let's grow together as a community of passionate coders!

© 2025-26 SYNTAX ERROR. All Rights Reserved.

This content is exclusively created by SYNTAX ERROR. Unauthorized copying, sharing, or redistribution of this DSA & Tech content is strictly prohibited. Protecting intellectual property ensures quality learning for everyone and maintains the integrity of this work.